LO4 — Kubernetes Containers

Accelerated Delivery of Multilayer Applications Using Containers

Complete Reference Guide — 53 Questions & Answers

Deployments & Scaling (Q1–Q15)

Q1 — Create a Deployment imperatively and export its YAML

Create the Deployment with a single imperative command:

```
kubectl create deployment web --image=nginx:1.25 --replicas=3
```

Export the generated YAML to a file (use --dry-run to capture the manifest Kubernetes would apply):

```
kubectl get deployment web -o yaml > web-deployment.yaml
```

Or generate without applying first:

```
kubectl create deployment web --image=nginx:1.25 --replicas=3 --dry-run=client -o yaml > web-deployment.yaml
```

The resulting YAML will include metadata, spec.replicas: 3, and the pod template with the nginx:1.25 container. Kubernetes adds fields like creationTimestamp, resourceVersion, and uid after the object is created.

Q2 — Authenticate to DockerHub to avoid pull-rate limits

You need to create a Secret of type docker-registry (also called an image-pull Secret). DockerHub applies rate limits to anonymous pulls (100/6h for free accounts). Authenticated pulls raise that limit significantly.

```
kubectl create secret docker-registry dockerhub-creds \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=<YOUR_USER> \
  --docker-password=<YOUR_TOKEN> \
  --docker-email=<YOUR_EMAIL>
```

Then reference it in the Deployment pod spec:

```
spec:
  imagePullSecrets:
    - name: dockerhub-creds
```

Kubernetes stores the credentials base64-encoded inside the Secret and passes them to the container runtime (containerd/CRI-O) when pulling each image.

Q3 — Scale web from 3 to 5 replicas two ways

Method 1 — kubectl scale (imperative):

```
kubectl scale deployment web --replicas=5
```

Method 2 — edit the manifest (declarative):

```
kubectl edit deployment web
```

In the editor, change spec.replicas from 3 to 5, save and quit. Alternatively patch the file and apply:

```
# In web-deployment.yaml set replicas: 5
kubectl apply -f web-deployment.yaml
```

After scaling, inspect the resulting ReplicaSet:

```
kubectl get replicasets -l app=web
```

You will see one ReplicaSet (e.g. web-6d4cf56db) with DESIRED=5, CURRENT=5, READY=5. Kubernetes creates a new ReplicaSet only when the pod template changes; scaling just updates the replica count on the existing RS.

Q4 — Rolling update from nginx:1.25 to nginx:1.27

```
kubectl set image deployment/web nginx=nginx:1.27
kubectl rollout status deployment/web
```

kubectl rollout status streams events until all pods are updated. Expected output:

```
Waiting for deployment "web" rollout to finish: 2 out of 5 new replicas have been
updated...
Waiting for deployment "web" rollout to finish: 3 old replicas are pending termination...
deployment "web" successfully rolled out
```

During a rolling update Kubernetes creates a new ReplicaSet, gradually scales it up while scaling the old one down, respecting maxSurge and maxUnavailable settings (default 25% each).

Q5 — Rollout history and rollback

```
kubectl rollout history deployment/web
```

Output example:

```
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

The CHANGE-CAUSE column shows an annotation you explicitly set. By default it is empty. To populate it:

```
# Option A: annotate after the change
kubectl annotate deployment/web kubernetes.io/change-cause="Upgraded to nginx:1.27"
# Option B: use --record flag (deprecated but still works in older clusters)
kubectl set image deployment/web nginx=nginx:1.27 --record
```

Roll back to the previous revision:

```
kubectl rollout undo deployment/web
```

Roll back to a specific revision:

```
kubectl rollout undo deployment/web --to-revision=1
```

Kubernetes restores the pod template from the stored ReplicaSet of that revision and performs a new rolling update back to it.

Q6 — RollingUpdate with maxSurge:1 and maxUnavailable:0

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Effect on availability:

- maxUnavailable: 0 means no existing pods are taken down until a replacement is healthy and ready. At no point during the update will the number of available pods drop below the desired count.
- maxSurge: 1 means Kubernetes can spin up at most one extra pod above the desired count at a time. This keeps memory/CPU overhead minimal.
- Combined: the update proceeds one pod at a time — add 1 new, wait until it passes readiness, remove 1 old — ensuring zero-downtime but a slightly slower rollout compared to higher surge values.

Q7 — Recreate strategy and when it is required

```
spec:
  strategy:
    type: Recreate
```

With Recreate, Kubernetes terminates ALL existing pods before creating any new ones. There is a downtime window between the old version going away and the new version becoming ready.

Concrete scenario where Recreate is required:

A legacy monolith uses a file-based lock (e.g., a SQLite database or a shared PID file on a hostPath or PVC). The application explicitly refuses to start if it detects another instance already holds the lock. Running two versions simultaneously — as RollingUpdate would — would cause the new pod to crash-loop immediately because the old pod still holds the lock. Using Recreate guarantees the old pod releases the lock before the new pod attempts to acquire it.

Other common scenarios: schema-level database migrations that are not backward-compatible, stateful applications with exclusive resource ownership, or licensing systems that permit only a single running instance.

Q8 — CPU/memory requests and limits

```
spec:
  template:
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "250m"
              memory: "256Mi"
```

Apply, then verify inside a running pod:

```
kubectl apply -f web-deployment.yaml
kubectl get pod <POD_NAME> -o jsonpath='{.spec.containers[0].resources}'
```

Or inspect with describe:

```
kubectl describe pod <POD_NAME> | grep -A6 'Limits\|Requests'
```

Requests are used by the scheduler to find a node with enough free capacity. Limits are enforced by the kubelet: a container exceeding its CPU limit is throttled; exceeding memory limit causes an OOMKill.

Q9 — revisionHistoryLimit and its effect on rollback

```
spec:
  revisionHistoryLimit: 3
```

Kubernetes stores each previous pod template as an inactive ReplicaSet. revisionHistoryLimit controls how many of these old ReplicaSets are retained.

- With revisionHistoryLimit: 3, Kubernetes keeps the 3 most recent old ReplicaSets (plus the currently active one). Older ones are garbage-collected.
- You can only roll back to a revision whose ReplicaSet still exists. If a revision was pruned, that rollback target is gone.
- The default value is 10. Setting it to 0 deletes all old ReplicaSets immediately, preventing any rollback.
- Setting it to 3 is a common production trade-off: enough history for quick recovery without storing unbounded objects in etcd.

```
kubectl get replicasets -l app=web
```

You will see at most 4 ReplicaSets total: 1 active + 3 historical.

Q10 — Stuck rollout with a non-existent image tag

```
kubectl set image deployment/web nginx=nginx:99.99.99-doesnotexist
kubectl rollout status deployment/web
```

The rollout will hang with a message like:

```
Waiting for deployment "web" rollout to finish: 1 out of 5 new replicas have been
updated...
```

Why old pods keep serving traffic:

- Kubernetes creates a new ReplicaSet and attempts to start one new pod. That pod enters ImagePullBackOff because the image tag does not exist in the registry.
- Because maxUnavailable defaults to 25% (rounded down), Kubernetes will not terminate more old pods than this threshold allows. The old pods remain Running and continue to serve requests.
- The Deployment controller waits for the new pod to become Ready before proceeding. Since it never will, the rollout stalls indefinitely.
- This is precisely the safety guarantee of RollingUpdate: a bad image cannot accidentally take down working pods.

Recover by rolling back:

```
kubectl rollout undo deployment/web
```

Q11 — Label selectors and the Deployment → ReplicaSet → Pod chain

```
# List pods belonging to the web Deployment
kubectl get pods -l app=web
# Or using the more specific selector from the RS
kubectl get pods -l app=web,pod-template-hash=<HASH>
```

Selector relationship:

- Deployment spec.selector.matchLabels (e.g. app: web) selects the ReplicaSets it owns by matching RS labels.
- ReplicaSet spec.selector.matchLabels (e.g. app: web, pod-template-hash: <hash>) selects pods it owns. Kubernetes automatically injects pod-template-hash to distinguish pods of different revisions.
- Pod labels must be a superset of the selector for the ReplicaSet to claim ownership.

Inspect the chain:

```
kubectl get deployment web -o jsonpath='{.spec.selector}'
kubectl get replicaset -l app=web -o jsonpath='{.items[*].spec.selector}'
kubectl get pods -l app=web --show-labels
```

Q12 — Expose a Deployment and what gets created

```
kubectl expose deployment web --port=80 --target-port=80 --name=web-svc
```

What was created: a Service object of type ClusterIP (the default). Verify:

```
kubectl get service web-svc
kubectl describe service web-svc
```

How the selector was derived:

kubectl expose copies the Deployment's spec.selector.matchLabels (app: web) directly into the Service's spec.selector. The Service then forwards traffic to any pod that has a matching app=web label — which are exactly the pods managed by the Deployment.

The Service gets a stable virtual IP (ClusterIP) inside the cluster. kube-proxy programs iptables/IPVS rules on every node to DNAT packets destined for ClusterIP:80 to one of the ready pod IPs.

Q13 — Sidecar container and shared network/volumes

```
spec:
  template:
    spec:
      volumes:
        - name: shared-logs
          emptyDir: {}
      containers:
        - name: nginx
          image: nginx:1.25
          volumeMounts:
            - name: shared-logs
              mountPath: /var/log/nginx
        - name: log-shipper
          image: busybox
          command: ["sh","-c","tail -f /logs/access.log"]
          volumeMounts:
            - name: shared-logs
              mountPath: /logs
```

How containers share network and volumes:

- Network: All containers in a pod share a single network namespace created by the pause (infra) container. They communicate via localhost and see the same port space. If nginx listens on :80, the sidecar can reach it at localhost:80.
- Volumes: Volumes are mounted by the kubelet into the pod's cgroup namespace. Both containers mount the same emptyDir at different paths, so writes by nginx to /var/log/nginx are immediately visible to the sidecar at /logs.
- Lifecycle: Both containers start simultaneously (unless an initContainer is used). If either crashes, the kubelet restarts only that container, not the whole pod.

Q14 — httpd Deployment with nodeSelector

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: httpd-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: httpd
  template:
    metadata:
      labels:
        app: httpd
    spec:
      nodeSelector:
        disktype: ssd
      containers:
        - name: httpd
          image: httpd:2.4
          ports:
            - name: http
              containerPort: 80
```

Why it stays Pending:

The scheduler filters nodes based on nodeSelector. If no node has the label disktype=ssd, the filter returns an empty candidate list and the pod cannot be placed. It enters Pending state indefinitely with the event: 0/1 nodes are available: 1 node(s) didn't match Pod's node affinity/selector.

Fix: label a node and the pods will schedule immediately.

```
kubectl label node <NODE_NAME> disktype=ssd
```

Q15 — Bare Pod vs Deployment-managed Pod on deletion

```
# Create a bare Pod
kubectl run sleep-pod --image=busybox --restart=Never -- sleep 3600
```

Behavior on deletion:

- Bare Pod (restart=Never): When you run kubectl delete pod sleep-pod, the pod is terminated and gone permanently. No controller watches it, so nothing recreates it. This is also true if the pod crashes — it stays in a terminal state.
- Deployment-managed Pod: When you delete a pod owned by a Deployment (e.g. kubectl delete pod web-xxx-yyy), the ReplicaSet controller detects that the actual replica count (N-1) is below the desired count (N) and immediately creates a replacement pod. The pod is effectively immortal as long as the Deployment exists.

This is the fundamental value of controllers: they reconcile actual state toward desired state continuously.

Q16 — StatefulSet for Redis with headless Service

```
apiVersion: v1
kind: Service
metadata:
  name: redis-headless
spec:
  clusterIP: None
  selector:
    app: redis
  ports:
```

```
    - port: 6379
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis-headless
  replicas: 3
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7
          ports:
            - containerPort: 6379
```

Stable pod names and DNS:

```
redis-0   # pod name
redis-1
redis-2
```

Per-pod DNS A records (via the headless Service):

```
redis-0.redis-headless.<namespace>.svc.cluster.local
redis-1.redis-headless.<namespace>.svc.cluster.local
redis-2.redis-headless.<namespace>.svc.cluster.local
```

The headless Service (clusterIP: None) tells CoreDNS to return individual pod IPs rather than a single VIP, enabling clients to target specific replicas — essential for leader election and replication configuration.

Q17 — DaemonSet and why exactly one pod runs per node

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: busybox-agent
spec:
  selector:
    matchLabels:
      app: busybox-agent
  template:
    metadata:
      labels:
        app: busybox-agent
    spec:
      containers:
        - name: agent
          image: busybox
```

```
          command: ["sleep", "infinity"]
```

Why exactly one pod per node:

The DaemonSet controller watches the set of schedulable nodes. For every node that matches its optional nodeSelector/tolerations (by default, all nodes), it ensures exactly one pod of that DaemonSet is running. If a new node joins the cluster, a pod is automatically created on it. If a node is removed, the pod is garbage-collected. If the pod is deleted, it is immediately recreated. This makes DaemonSets ideal for per-node infrastructure agents like log collectors (Fluentd), metrics agents (node-exporter), or CNI plugins.

Q18 — Job that computes once and completes

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-calculator
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: pi
          image: perl:5.34
          command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(100)"]
```

Run and check completions:

```
kubectl apply -f job.yaml
kubectl get jobs
# Output: NAME            COMPLETIONS   DURATION   AGE
#         pi-calculator   1/1           8s         30s
```

Read the result:

```
kubectl logs job/pi-calculator
```

A Job's restartPolicy must be Never or OnFailure (never Always). When the container exits 0, the pod phase becomes Succeeded and COMPLETIONS shows 1/1.

Q19 — CronJob printing the date every minute

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: print-date
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: date
              image: busybox
              command: ["sh", "-c", "date"]
```

Suspend the CronJob (no new Jobs will fire):

```
kubectl patch cronjob print-date -p '{"spec":{"suspend":true}}'
```

List Jobs spawned by the CronJob:

```
kubectl get jobs -l app=print-date
# Or by owner reference
kubectl get jobs --selector=job-name
```

The CronJob controller creates a new Job object at each scheduled interval. Each Job in turn creates a Pod. You can control how many successful/failed Job histories are retained via successfulJobsHistoryLimit and failedJobsHistoryLimit.

Q20 — Manually trigger a Job from a CronJob

```
kubectl create job print-date-manual --from=cronjob/print-date
```

This creates a one-off Job using the exact same jobTemplate from the CronJob, ignoring the schedule. Useful for:

- Testing a CronJob's logic on demand before enabling the schedule.
- Re-running a failed or missed execution.
- Debugging the container command in isolation.

```
kubectl get jobs
kubectl logs job/print-date-manual
```

Q21 — StatefulSet ordered creation/termination vs Deployment

StatefulSet pod creation order:

```
kubectl apply -f redis-statefulset.yaml
kubectl get pods -w
# redis-0   0/1   ContainerCreating   ...
# redis-0   1/1   Running             ...   <- must be Running before redis-1 starts
# redis-1   0/1   ContainerCreating   ...
# redis-1   1/1   Running             ...
# redis-2   0/1   ContainerCreating   ...
```

StatefulSet termination order (scale-down or delete) is reverse: redis-2 first, then redis-1, then redis-0.

Contrast with Deployment:

- Deployments create all replicas in parallel. Pod names are random hashes (web-6d4cf56db-xk9pb). There is no guaranteed ordering of creation, termination, or scheduling.
- StatefulSets provide: stable pod identity (redis-0), stable network identity (DNS), stable storage (one PVC per pod), and ordered operations. These guarantees are essential for clustered databases and distributed systems.

Q22 — Two containers sharing emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-vol-demo
spec:
  volumes:
    - name: shared
      emptyDir: {}
  containers:
    - name: writer
      image: busybox
      command: ["sh","-c","echo hello-from-writer > /data/msg.txt && sleep 3600"]
      volumeMounts:
```

```
        - name: shared
          mountPath: /data
    - name: reader
      image: busybox
      command: ["sh","-c","sleep 5 && cat /shared/msg.txt && sleep 3600"]
      volumeMounts:
        - name: shared
          mountPath: /shared
```

Prove the volume is shared:

```
kubectl apply -f shared-vol-demo.yaml
kubectl logs shared-vol-demo -c reader
# Output: hello-from-writer
```

emptyDir is created on the node when the pod is assigned. It lives for the lifetime of the pod and is deleted when the pod is removed. Both containers mount it at different paths but it is the same directory on the node filesystem.

Q23 — List StorageClasses, PVs, and PVCs

```
kubectl get storageclass
kubectl get persistentvolumes
kubectl get persistentvolumeclaims --all-namespaces
```

On a minikube cluster, you will typically see:

- StorageClass: standard (provisioner: k8s.io/minikube-hostpath, default)
- PVs: persistent volumes created either statically by an admin or dynamically by the provisioner when a PVC is created.
- PVCs: claims made by pods/StatefulSets. The STATUS column shows Bound (matched to a PV), Pending (waiting), or Lost.

The binding process: a PVC specifies storageClassName, accessModes, and storage size. The provisioner creates a matching PV and binds them together. Once bound, the PVC is exclusively tied to that PV until released.

Q24 — Mount a ConfigMap as a volume

```
kubectl create configmap nginx-conf \
  --from-literal=index.html='<h1>Hello K8s</h1>' \
  --from-literal=info.txt='Kubernetes ConfigMap demo'
---
spec:
  volumes:
    - name: config-vol
      configMap:
        name: nginx-conf
  containers:
    - name: nginx
      image: nginx:1.25
      volumeMounts:
        - name: config-vol
          mountPath: /etc/config
```

Each ConfigMap key becomes a file in /etc/config:

```
kubectl exec <POD> -- ls /etc/config
# index.html  info.txt
kubectl exec <POD> -- cat /etc/config/index.html
```

```
# <h1>Hello K8s</h1>
```

When the ConfigMap is updated (kubectl edit configmap nginx-conf), Kubernetes syncs the mounted files after a short delay (kubelet sync period, typically ~1 minute). The application must re-read the files to pick up changes.

Q25 — Mount a single ConfigMap key with subPath

```
volumeMounts:
  - name: config-vol
    mountPath: /etc/nginx/conf.d/default.conf
    subPath: nginx.conf
```

When to use subPath:

- Without subPath, mounting a ConfigMap at /etc/nginx/conf.d/ replaces the entire directory with only the ConfigMap keys. Existing files in that directory are hidden.
- With subPath, only the specified key (nginx.conf) is mounted at the exact target path, leaving all other files in /etc/nginx/conf.d/ untouched.

Important update caveat:

Files mounted via subPath are NOT automatically updated when the ConfigMap changes. The container must be restarted (or the pod replaced) to pick up new values. This is a known Kubernetes limitation: only whole-volume ConfigMap mounts benefit from the automatic sync.

Q26 — Mount a Secret as a volume

```
kubectl create secret generic db-creds \
  --from-literal=username=admin \
  --from-literal=password=s3cr3t
---
spec:
  volumes:
    - name: secret-vol
      secret:
        secretName: db-creds
        defaultMode: 0400
  containers:
    - name: app
      image: busybox
      command: ["sleep","3600"]
      volumeMounts:
        - name: secret-vol
          mountPath: /etc/secrets
          readOnly: true
```

Verify decoded values and permissions:

```
kubectl exec <POD> -- cat /etc/secrets/username    # admin (plain text, already decoded)
kubectl exec <POD> -- cat /etc/secrets/password    # s3cr3t
kubectl exec <POD> -- ls -la /etc/secrets/         # -r-------- (0400)
```

Kubernetes stores the secret in etcd as base64. When mounted as a volume, the kubelet decodes it to plain text and places the files in a tmpfs (in-memory filesystem) inside the pod, so the decoded values never touch node disk storage. The 0400 permissions ensure only the container's user can read them.

Q27 — StatefulSet PVCs on scale and deletion

When you scale a StatefulSet, Kubernetes automatically creates one PVC per new replica using the volumeClaimTemplates:

```
kubectl scale statefulset redis --replicas=5
kubectl get pvc
# data-redis-0  Bound  ...
# data-redis-1  Bound  ...
# ...
# data-redis-4  Bound  ...
```

What happens to PVCs when the StatefulSet is deleted:

PVCs are NOT deleted when the StatefulSet is deleted. This is intentional: the PVCs (and their underlying PVs) contain stateful data (database files, etc.) that must survive controller restarts. If you want to remove the data you must delete the PVCs manually:

```
kubectl delete pvc -l app=redis
```

Scaling down (e.g. back to 3) also does not delete the PVCs for pods redis-3 and redis-4. If you scale back up to 5, those same PVCs are reattached to the new pods, preserving state.

Q28 — initContainer to pre-populate a shared volume

```
spec:
  volumes:
    - name: init-data
      emptyDir: {}
  initContainers:
    - name: data-loader
      image: busybox
      command: ["sh","-c","echo 'pre-loaded data' > /data/seed.txt"]
      volumeMounts:
        - name: init-data
          mountPath: /data
  containers:
    - name: main-app
      image: busybox
      command: ["sh","-c","cat /app-data/seed.txt && sleep 3600"]
      volumeMounts:
        - name: init-data
          mountPath: /app-data
```

initContainers run to completion before any main container starts. The shared volume (emptyDir) persists across all containers in the pod's lifetime. The main container reads seed.txt that was written by the initContainer. This pattern is common for database schema setup, config rendering, or artifact downloading before the app starts.

Q29 — readOnly volume mount

```
volumeMounts:
  - name: config-vol
    mountPath: /etc/config
    readOnly: true
```

Prove writes are rejected:

```
kubectl exec <POD> -- touch /etc/config/newfile
# touch: /etc/config/newfile: Read-only file system
kubectl exec <POD> -- rm /etc/config/index.html
# rm: can't remove '/etc/config/index.html': Read-only file system
```

The kernel mounts the volume with the MS_RDONLY flag. Any write syscall (open with O_WRONLY, unlink, rename) returns EROFS (Read-only file system). This is critical for ConfigMaps and Secrets that should not be modified by the container — accidental writes do not alter the Kubernetes objects but would be silently dropped without readOnly:true, which could cause subtle bugs.

Q30 — emptyDir sizeLimit

```
volumes:
  - name: scratch
    emptyDir:
      sizeLimit: 100Mi
```

What happens if the container exceeds the limit:

The kubelet's eviction manager monitors disk usage of emptyDir volumes. If a container writes more than 100Mi to the volume, the kubelet evicts the pod. The pod is then restarted (if it has a restart policy of Always or OnFailure), starting with an empty volume again.

Important notes:

- sizeLimit does not enforce a hard block at the OS level like a quota. It is checked periodically by the kubelet (default every 10s). A container can briefly exceed the limit between checks.
- If medium: Memory is set on the emptyDir, it is backed by tmpfs. The sizeLimit then also maps to a memory limit for the pod (counted against container memory limits).
- Without sizeLimit, an emptyDir is bounded only by the node's available disk space.

Q31 — Generic Secret from literals

```
kubectl create secret generic db-credentials \
  --from-literal=username=myuser \
  --from-literal=password=mypassword123
```

Inspect the stored values:

```
kubectl get secret db-credentials -o yaml
# data:
#   password: bXlwYXNzd29yZDEyMw==
#   username: bXl1c2Vy
```

Decode to confirm:

```
echo 'bXl1c2Vy' | base64 -d    # myuser
echo 'bXlwYXNzd29yZDEyMw==' | base64 -d  # mypassword123
```

Why base64 is not encryption:

base64 is an encoding scheme, not encryption. Anyone with kubectl get secret permissions can decode the values instantly. Kubernetes Secrets are only as secure as your RBAC policies and etcd encryption-at-rest configuration. For true secrets management, use tools like HashiCorp Vault, AWS Secrets Manager, or Sealed Secrets with etcd encryption enabled.

Q32 — Secret from files (--from-file)

```
# Assume you have files tls.crt and tls.key
kubectl create secret generic tls-files \
  --from-file=tls.crt=./tls.crt \
  --from-file=tls.key=./tls.key
```

Inspect the resulting keys:

```
kubectl get secret tls-files -o jsonpath='{.data}' | python3 -m json.tool
# {
#   "tls.crt": "<base64-encoded cert>",
```

```
#   "tls.key": "<base64-encoded key>"
# }
```

The resulting keys are named after the source filenames (tls.crt and tls.key). If you use --from-file=./tls.crt without specifying a key name, the key is the filename. With --from-file=mycert=./tls.crt the key becomes mycert. This Secret type is generic; for a properly typed TLS Secret, use the kubernetes.io/tls type (Q33).

Q33 — kubernetes.io/tls typed Secret

```
kubectl create secret tls my-tls-secret \
  --cert=./tls.crt \
  --key=./tls.key
```

The result is a Secret with type: kubernetes.io/tls and exactly two keys: tls.crt and tls.key. Kubernetes validates that these keys are present when you use this type.

Where it is consumed:

- Ingress controllers (nginx, Traefik, etc.) reference this Secret in Ingress resources to terminate HTTPS: spec.tls[].secretName: my-tls-secret.
- Applications that need to perform mutual TLS authentication can mount it as a volume to present a client certificate.
- Service mesh proxies (Istio, Linkerd) can read it for workload identity.

Using the typed Secret ensures tooling and admission webhooks can validate the format before the certificate is used in production.

Q34 — Secret as volume vs environment variables: security trade-offs

Volume mount approach:

```
volumeMounts:
  - name: secret-vol
    mountPath: /etc/secrets
    readOnly: true
```

Environment variable approach:

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: db-credentials
        key: password
```

Security trade-offs:

- Volume mounts: Secrets are stored in tmpfs (never on disk). Files can be updated automatically when the Secret changes. The process must actively read the file — it cannot be accidentally logged. Preferred for credentials that rotate.
- Environment variables: Easier to use (no file I/O). However, env vars are visible in /proc/<pid>/environ, can be dumped with kubectl exec -- env, are often printed by crash reporters and logging frameworks, and are not updated when the Secret changes (pod restart required). Less secure.
- Both: Neither approach encrypts the value in memory once the process has it. Both are equally protected (or unprotected) against etcd compromise without encryption-at-rest.

Q35 — envFrom with secretRef

```
spec:
```

```
  containers:
    - name: app
      image: nginx:1.25
      envFrom:
        - secretRef:
            name: db-credentials
```

Every key in the Secret (e.g. username, password) becomes an environment variable with the same name. Verify:

```
kubectl exec <POD> -- env | grep -E 'username|password'
# username=myuser
# password=mypassword123
```

envFrom is convenient when a Secret contains many keys you want available without listing each one. You can combine secretRef and configMapRef in the same envFrom array. Use the prefix field to namespace the variables:

```
envFrom:
  - secretRef:
      name: db-credentials
    prefix: DB_
# Results in DB_username, DB_password
```

Q36 — docker-registry Secret and imagePullSecrets

```
kubectl create secret docker-registry my-registry-creds \
  --docker-server=registry.example.com \
  --docker-username=ci-bot \
  --docker-password=<TOKEN>
---
spec:
  imagePullSecrets:
    - name: my-registry-creds
  containers:
    - name: app
      image: registry.example.com/myapp:v1
```

When it is required:

- Pulling from a private registry that requires authentication (ECR, GCR, Docker Hub private repos, self-hosted Harbor).
- DockerHub authenticated pulls to avoid the 100-pulls/6h anonymous rate limit.
- Air-gapped environments with an internal registry that requires credentials.

Without imagePullSecrets pointing to valid credentials, the kubelet returns ErrImagePull / ImagePullBackOff and the pod never starts. You can also attach the Secret to a ServiceAccount so all pods using that SA inherit the pull credentials automatically.

Q37 — Mount only one key from a multi-key Secret

```
volumes:
  - name: partial-secret
    secret:
      secretName: db-credentials
      items:
        - key: password
          path: db-password
volumeMounts:
```

```
  - name: partial-secret
    mountPath: /run/secrets
    readOnly: true
```

Only /run/secrets/db-password will exist inside the container. The username key is not projected. The items list lets you:

- Choose which keys to expose (principle of least privilege).
- Rename keys to friendly filesystem paths (key: tls.crt, path: cert.pem).
- Control per-file permissions with mode: 0400 on each item.

Q38 — ClusterIP Service and DNS resolution

```
kubectl expose deployment web --port=80 --name=web-clusterip
```

From another pod, resolve it by DNS:

```
kubectl run debug --rm -it --image=busybox -- sh
# Inside the shell:
nslookup web-clusterip.default.svc.cluster.local
# Server:    10.96.0.10
# Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local
# Name:      web-clusterip.default.svc.cluster.local
# Address 1: 10.100.x.y
```

DNS format: <service>.<namespace>.svc.<cluster-domain>. Within the same namespace you can use just web-clusterip. CoreDNS resolves Service names to their ClusterIP. kube-proxy then handles the DNAT from ClusterIP to an actual pod IP based on the Service's Endpoints.

Q39 — NodePort Service

```
kubectl expose deployment web --port=80 --type=NodePort --name=web-nodeport
kubectl get service web-nodeport
# NAME           TYPE       CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
# web-nodeport   NodePort   10.96.x.x     <none>        80:31245/TCP   5s
```

Access via minikube:

```
minikube service web-nodeport --url
# http://192.168.49.2:31245
curl http://$(minikube ip):31245
```

NodePort allocates a port in the range 30000–32767 on every node. kube-proxy on each node forwards traffic from <node-ip>:<nodePort> to the Service's ClusterIP, which then load-balances to pods. In production, a LoadBalancer or Ingress sits in front so external clients hit a stable IP rather than individual node IPs.

Q40 — LoadBalancer Service and minikube tunnel

```
kubectl expose deployment web --port=80 --type=LoadBalancer --name=web-lb
kubectl get service web-lb
# NAME     TYPE           CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
# web-lb   LoadBalancer   10.96.x.x     <pending>     80:31xxx/TCP   10s
```

Why EXTERNAL-IP is pending on minikube:

A LoadBalancer Service normally triggers a cloud provider's controller (AWS ELB, GCP GLB, Azure LB) to provision an external IP. Minikube has no cloud provider, so the IP stays in status Pending indefinitely.

Get access with minikube tunnel:

```
minikube tunnel   # run in a separate terminal (requires sudo on some platforms)
```

```
kubectl get service web-lb
# EXTERNAL-IP is now 127.0.0.1
curl http://127.0.0.1:80
```

minikube tunnel creates a network route that forwards the LoadBalancer's IP to the minikube VM, emulating what a cloud controller would do.

Q41 — Headless Service for StatefulSet and per-pod DNS

```
apiVersion: v1
kind: Service
metadata:
  name: redis-headless
spec:
  clusterIP: None   # headless
  selector:
    app: redis
  ports:
    - port: 6379
```

With clusterIP: None, CoreDNS registers an A record for each ready pod instead of a single VIP:

```
nslookup redis-headless.default.svc.cluster.local
# Returns: 3 A records (one per pod IP)
nslookup redis-0.redis-headless.default.svc.cluster.local
# Returns: the specific IP of redis-0 only
```

This is essential for StatefulSets because clients (e.g. Redis Sentinel, Kafka brokers) need to address individual pods by stable identity, not load-balance across them blindly.

Q42 — Service Endpoints and how they're populated

```
kubectl get endpoints web-clusterip
# NAME            ENDPOINTS                         AGE
# web-clusterip   10.244.0.5:80,10.244.0.6:80,...   5m
kubectl get endpointslices -l kubernetes.io/service-name=web-clusterip
```

How Endpoints are populated:

The Endpoints controller watches Pods and Services. When a Pod's labels match a Service's spec.selector AND the Pod is Ready (readiness probe passes), the controller adds pod-ip:containerPort to the Endpoints object. When a pod fails its readiness probe or is deleted, its entry is removed. kube-proxy watches Endpoints and updates its iptables/IPVS rules so that only healthy pods receive traffic. This is why failing readiness probes removes a pod from rotation without killing it.

Q43 — Cross-namespace DNS: FQDN works, short name fails

```
# From a pod in namespace 'staging':
nslookup web-clusterip.default.svc.cluster.local
# Works — resolves to the ClusterIP in 'default' namespace
nslookup web-clusterip
# Fails — CoreDNS searches staging.svc.cluster.local first, then cluster.local
# 'web-clusterip' expands to 'web-clusterip.staging.svc.cluster.local' — not found
```

CoreDNS uses the pod's /etc/resolv.conf search domains: <pod-namespace>.svc.cluster.local, svc.cluster.local, cluster.local. A short name is searched within the current namespace first. To reach a Service in another namespace you must use the full qualified name: <service>.<namespace>.svc.cluster.local.

Q44 — kubectl port-forward to Service vs to Pod

```
# Forward to Service (traffic goes to one of the Service's pods)
kubectl port-forward service/web-clusterip 8080:80
# Forward to specific Pod (bypasses Service, direct tunnel)
kubectl port-forward pod/web-abc123-xyz 8080:80
```

Differences and use cases:

- port-forward to Service: useful when you want to test a Service configuration (that selector is working, port mappings are correct). Still only opens one tunnel — it picks one pod, not load-balancing in any production sense.
- port-forward to Pod: useful for debugging a specific pod instance (tailing logs, checking internal state) bypassing Service selector. The connection goes directly to that pod via the API server's websocket proxy.
- Neither is appropriate for production traffic. Both tunnel through the Kubernetes API server, adding latency and limiting throughput. Use NodePort/LoadBalancer/Ingress for real traffic.

Q45 — Service port, targetPort, and nodePort

```
spec:
  type: NodePort
  ports:
    - port: 80          # ClusterIP port — what clients inside the cluster use
      targetPort: 8080  # Container port — what the application actually listens on
      nodePort: 31080   # Node port — what external clients hit on each node IP
```

Summary:

- port: the port on the Service's ClusterIP. Cluster-internal clients connect to clusterIP:port.
- targetPort: the port on the pod/container the traffic is forwarded to. Can be a number or named port (e.g. http). Defaults to the value of port if omitted.
- nodePort: (NodePort/LoadBalancer only) the port opened on every cluster node. External traffic hits <node-ip>:nodePort and is forwarded to port then targetPort.

Q46 — Verify connectivity to ClusterIP from a debug pod

```
kubectl run tmp --rm -it --image=busybox -- sh
# Inside the shell:
wget -qO- http://web-clusterip.default.svc.cluster.local
# Or with nc (netcat) to check TCP connectivity:
nc -zv web-clusterip.default.svc.cluster.local 80
# web-clusterip.default.svc.cluster.local (10.96.x.x:80) open
```

The --rm flag deletes the pod when you exit the shell. This throwaway debug pattern is extremely useful because it runs inside the cluster network, using the same DNS and network policies as production pods. Testing from your laptop (via port-forward) does not exercise the full path through kube-proxy and Service routing.

Q47 — One Service load-balancing across two Deployments

```
# Deployment A
kubectl create deployment app-v1 --image=nginx:1.25
kubectl label deployment app-v1 app=shared-app
kubectl label pods -l app=app-v1 app=shared-app
# Deployment B
kubectl create deployment app-v2 --image=nginx:1.27
kubectl label deployment app-v2 app=shared-app
kubectl label pods -l app=app-v2 app=shared-app
```

```
# One Service selecting both
kubectl expose deployment app-v1 --selector=app=shared-app --port=80 --name=shared-svc
```

The Service selector app=shared-app matches pods from both Deployments. The Endpoints object will list all matching pod IPs. Verify:

```
kubectl get endpoints shared-svc
# Should show IPs from both Deployments
```

This pattern is used for blue-green deployments and canary releases — gradually shifting traffic by adjusting replica counts on each Deployment.

Q48 — Systematic diagnosis when a pod can't reach a Service

Step-by-step diagnostic:

- 1. DNS: kubectl exec <POD> -- nslookup <service>.<namespace>.svc.cluster.local — does it resolve? If not, check CoreDNS pods in kube-system.
- 2. Endpoints: kubectl get endpoints <service> — are there any endpoints? Empty means no pod matched the selector or all pods are not Ready.
- 3. Selector labels: kubectl describe service <service> | grep Selector — then kubectl get pods --show-labels. Do the pod labels match the selector exactly?
- 4. Readiness: kubectl describe pod <POD> | grep -A10 Readiness — is the readiness probe passing? Failing readiness removes the pod from endpoints.
- 5. Port: kubectl describe service <service> | grep Port — is targetPort matching what the container actually listens on? kubectl exec <POD> -- netstat -tlnp | grep <port>.
- 6. NetworkPolicy: kubectl get networkpolicy — is there a policy blocking the traffic path?

Q49 — NodePort Service and verification

```
kubectl expose deployment web --type=NodePort --port=80 --name=web-np
kubectl get svc web-np
# PORT(S): 80:3xxxx/TCP
```

Verify it works:

```
NODE_PORT=$(kubectl get svc web-np -o jsonpath='{.spec.ports[0].nodePort}')
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[0].address}')
curl http://$NODE_IP:$NODE_PORT
# Returns nginx welcome page
```

On minikube, use minikube ip instead of the node address. The NodePort is accessible from any cluster node regardless of which node runs the pod — kube-proxy routes the packet if the pod is on a different node.

Q50 — HTTP livenessProbe on nginx Deployment

```
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 10
        periodSeconds: 10
        failureThreshold: 3
```

Verify via kubectl describe pod:

```
kubectl describe pod <POD_NAME> | grep -A10 'Liveness'
# Liveness: http-get http://:80/ delay=10s timeout=1s period=10s #success=1 #failure=3
```

If the probe fails 3 times consecutively, the kubelet restarts the container. The pod itself is not deleted — only the container inside it. The restart count is visible in kubectl get pods (RESTARTS column). livenessProbe is for detecting deadlocks or infinite loops — cases where the process is running but not functional.

Q51 — tcpSocket readiness probe for Redis

```
spec:
  containers:
    - name: redis
      image: redis:7
      readinessProbe:
        tcpSocket:
          port: 6379
        initialDelaySeconds: 5
        periodSeconds: 5
        failureThreshold: 3
```

Verify:

```
kubectl describe pod <REDIS_POD> | grep -A8 'Readiness'
# Readiness: tcp-socket :6379 delay=5s timeout=1s period=5s #success=1 #failure=3
```

A tcpSocket probe attempts a TCP connection to the specified port. If the connection succeeds (SYN/SYN-ACK), the probe passes. It does not send any data — suitable for protocols like Redis, MySQL, or any TCP server where a successful connection handshake implies readiness. Unlike httpGet, no HTTP response code is checked. The pod is only added to Service Endpoints once this probe passes.

Q52 — Startup probe for a slow-booting application

If an application takes up to 2 minutes to start:

```
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 24
  periodSeconds: 5
```

Justification: failureThreshold (24) × periodSeconds (5s) = 120 seconds maximum startup budget. If the app is not ready within 2 minutes, the kubelet kills it.

Why a startup probe instead of just a large initialDelaySeconds on livenessProbe:

- initialDelaySeconds is a static wait — even if the app starts in 10 seconds, liveness doesn't kick in until the delay passes.
- startupProbe is dynamic — it polls every 5 seconds and hands off to liveness/readiness the moment it first succeeds. Fast starts are rewarded with faster traffic.
- While startupProbe is active, liveness and readiness probes are disabled, preventing them from killing a legitimately slow-starting container.

Q53 — Default behavior with no probes defined

When no probes are defined:

- livenessProbe absent: The kubelet never restarts the container based on health. If the process enters a deadlock, hangs indefinitely, or stops responding, it will continue running until the

process itself crashes (exits non-zero) or you manually intervene. A crashed process is still restarted by the container restart policy.

- readinessProbe absent: The container is considered Ready immediately when it starts (specifically, as soon as it transitions to Running state). It is added to Service Endpoints right away, even if the application inside has not finished initializing and may return errors to clients.
- startupProbe absent: livenessProbe starts immediately at container start. If initialDelaySeconds is too short and the app is slow to boot, the liveness probe may kill the container before it finishes starting, causing a crash loop.

In summary: without probes, Kubernetes assumes your containers are healthy and ready from the moment they start. This is acceptable for development but dangerous in production — slow startups, deadlocks, and unhealthy pods can serve traffic or stay running indefinitely without automatic remediation.